



**Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026**

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

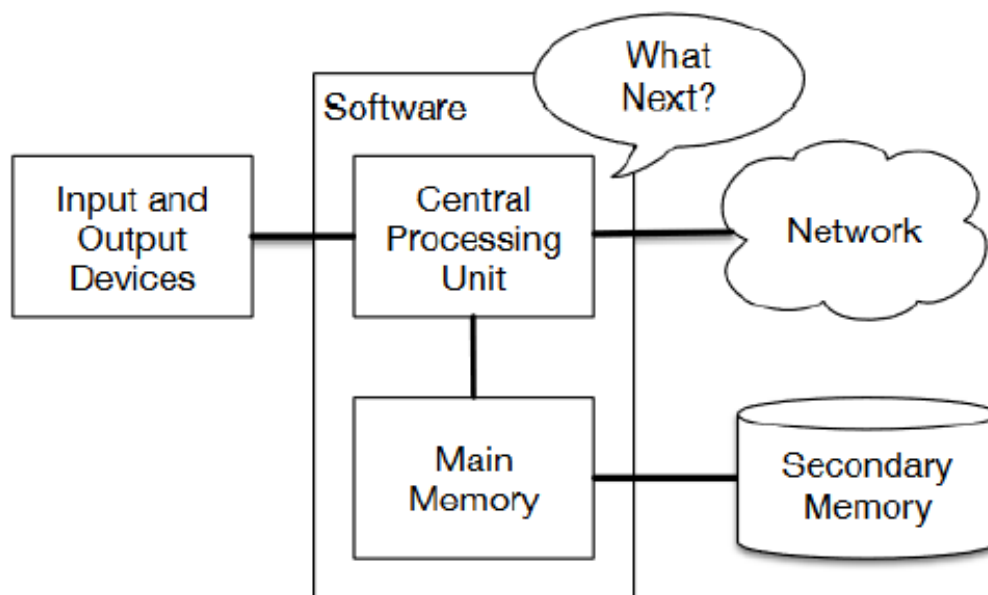
SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251233
Nama Lengkap	Mikael Gratianus Satrio Adi Kuncoro
Minggu ke / Materi	10 / Manajemen File

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

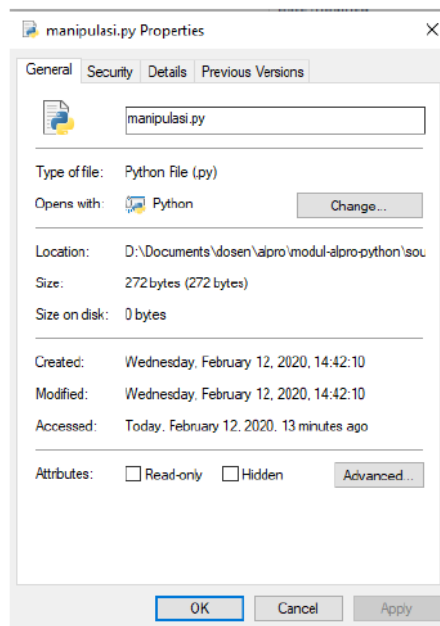
Pengantar File

Program yang berjalan membutuhkan memory primer di didalam komputer, semua data yang diprogram di simpan dan ketika program selesai dijalankan dan dimatikan, maka semua data yang ada didalam program tersebut juga ikut hilang. Ini berarti data yang tersimpan dalam memory sifatnya volatile. Karena itu program yang menggunakan memory primer tidak akan dapat menyimpan data setelah program dimatikan. Maka harus menggunakan penyimpanan tetap yaitu secondary memory.



Gambar 1.1: Secondary Memory di Komputer

File yang disimpan dalam secondary memory tidak akan hilang walaupun komputer dimatikan, karena pada dasarnya file adalah bit-bit data yang berupa kumpulan informasi yang saling berelasi satu sama lain sebagai satu kesatuan yang disimpan di dalam secondary memory secara permanen. File bisa berupa file system, file program(binary), file multimedia, file teks, dan lain sebagainya yang juga memiliki property seperti nama file, ukuran, letak di harddisk, owner, hak akses, tanggal akses, dan lain-lain. Contoh property sebuah file dapat dilihat dari Gambar 1.2.



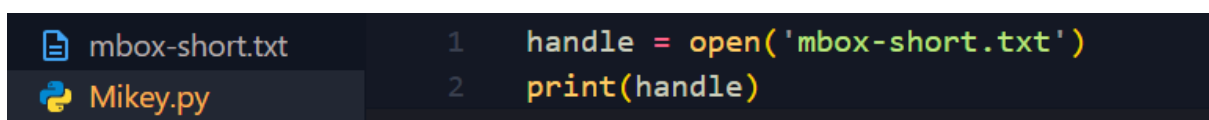
Gambar1.2: Contoh Property File

Pengaksesan File

Berikut merupakan langkah-langkah dalam mengakses file:

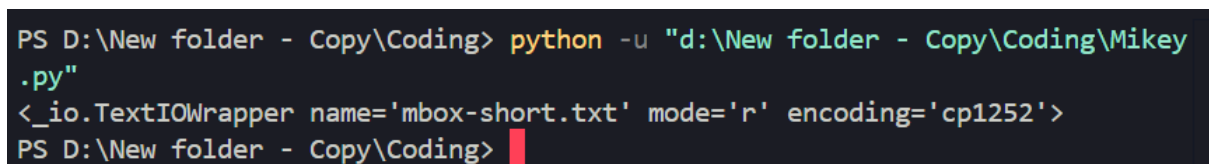
1. Menyiapkan file dan path yang akan diakses
2. Open file
3. Lakukan sesuatu dengan file tersebut, seperti ditampilkan (read) isinya atau diubah / ditulisi (write)
4. Close file

Program Python dapat dibuat sebagai berikut:



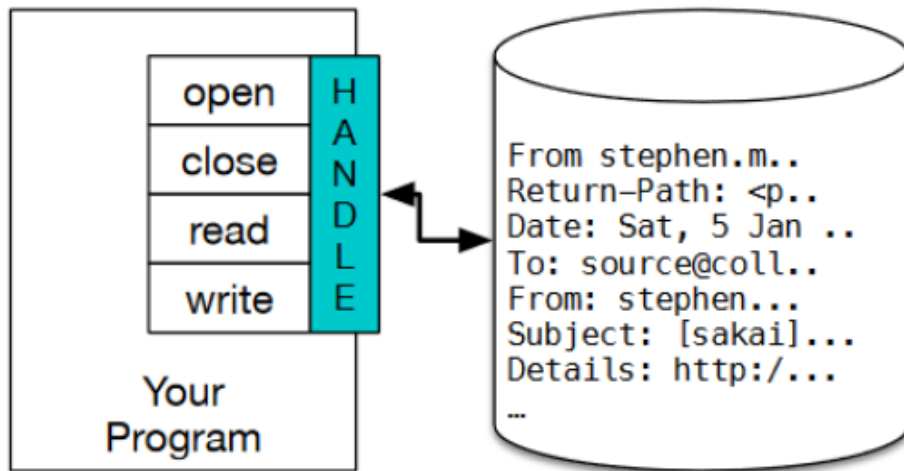
Gambar 1.3: Contoh program pengaksesan file pada python

Hasilnya adalah:



Gambar 1.4: Output program pengaksesan file

Berikut ini juga merupakan gambaran handle file:



Gambar 1.5: Ilustrasi Handle File

Jadi hasilnya berupa tampilan nama file, modenya (r = read), dan encoding yang digunakan yaitu unicode UTF-8 dari sistem io pada Python. Jika nama file tidak ada / tidak ditemukan, maka output akan error:

```
mbox-short.txt
Mikey.py
1 handle = open('tidak_ada.txt')
2 print(handle)

PROBLEMS  TERMINAL  ...  Code  +  -  [  ]  [  ]  ...  |

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\Mikey.py"
Traceback (most recent call last):
  File "d:\New folder - Copy\Coding\Mikey.py", line 1, in <module>
    handle = open('tidak_ada.txt')
FileNotFoundError: [Errno 2] No such file or directory: 'tidak_ada.txt'
PS D:\New folder - Copy\Coding>
```

Gambar 1.6: Contoh penerapan nya jika tidak ada file di dictionary

Pada file teks, biasa nya file akan terdiri dari baris demi baris string. Cara pembacaan file teks biasanya juga menggunakan model baca baris demi baris untuk setiap string yang ditemukan sampai dengan EOF (End of File). Berikut merupakan contoh tampilan baris-baris dari sebuah file teks mbox-short.txt:

```

1 From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
2 Return-Path: <postmaster@collab.sakaiproject.org>
3 Received: from murder (mail.umich.edu [141.211.14.90])
4 | by frankenstein.mail.umich.edu (Cyrus v2.3.8) with LMTPA;
5 | Sat, 05 Jan 2008 09:14:16 -0500
6 X-Sieve: CMU Sieve 2.3
7 Received: from murder ([unix socket])
8 | by mail.umich.edu (Cyrus v2.2.12) with LMTPA;
9 | Sat, 05 Jan 2008 09:14:16 -0500
0 Received: from holes.mr.itd.umich.edu (holes.mr.itd.umich.edu [14
1 | by flawless.mail.umich.edu () with ESMTP id m05EEFR1013674;

```

Gambar 1.7: Contoh File Teks dan Barisnya

Manipulasi File

Memanipulasi file harus dimulai dengan membaca file tersebut terlebih dahulu baru bisa dimanipulasi. Berikut cara membaca file pada python:

1. Siapkan file
2. Open file
3. Loop setiap baris pada file
4. Close file

Cara pembacaan file nya sebagai berikut:

```

1 handle = open('mbox-short.txt')
2 count = 0
3 for line in handle:
4 |     count = count + 1
5 print('Line Count:', count)

```

Gambar 1.8: Contoh program pembacaan file pada python

Hasilnya:

```

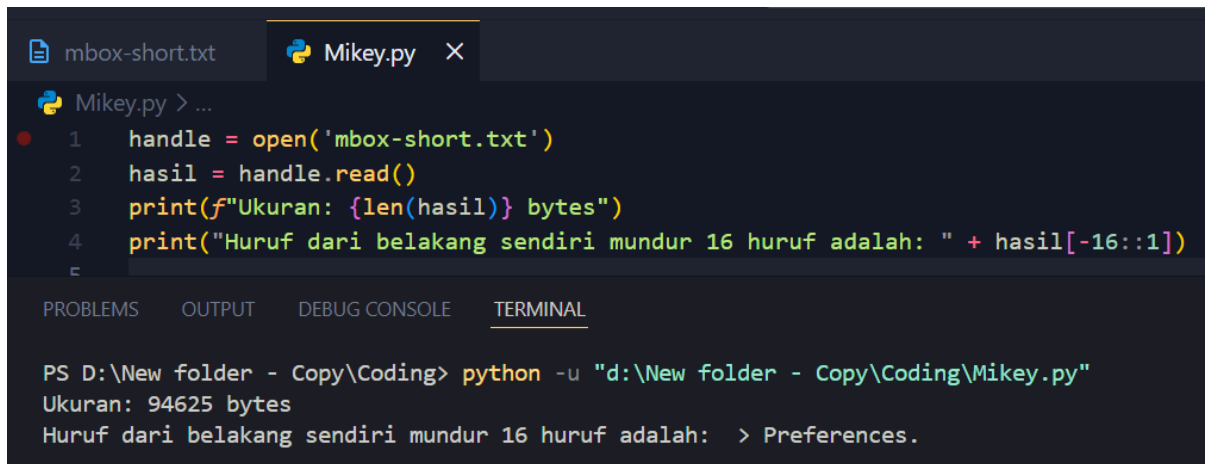
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\Mikey
.py"
Line Count: 1909
PS D:\New folder - Copy\Coding>

```

Gambar 1.9: Output program pembacaan file

Untuk mengatasi kemungkinan sistem membuka file yang besar sekali dalam satu waktu dan akhirnya hang / crash karena ukuran file yang sangat besar, maka file harus dibaca baris-perbaris.

Cara menampilkan ukuran file teks dalam bytes, dapat menggunakan fungsi `len` dari string yang ada pada file.

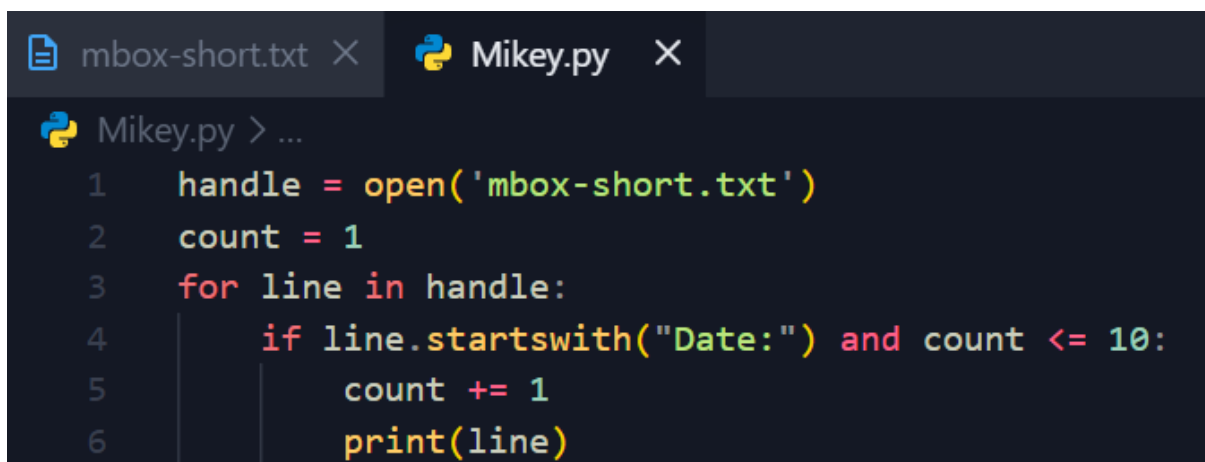


```
mbox-short.txt  Mikey.py X
Mikey.py > ...
1  handle = open('mbox-short.txt')
2  hasil = handle.read()
3  print(f"Ukuran: {len(hasil)} bytes")
4  print("Huruf dari belakang sendiri mundur 16 huruf adalah: " + hasil[-16::1])
5
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\Mikey.py"
Ukuran: 94625 bytes
Huruf dari belakang sendiri mundur 16 huruf adalah: > Preferences.
```

Gambar 2.0: Source code menampilkan ukuran file teks dalam bytes

Program ini akan membuka file `mbox-short.txt`, menampilkan ukuran berapa banyak huruf yang ada pada file tersebut (jika 1 karakter = 1 byte, maka bisa disebut juga ukuran berapa banyak karakter = ukuran file tersebut dalam byte), dan terakhir menampilkan string dari huruf paling belakang maju 16 huruf kedepan. Perintah `read()` sangat boros memory, jika ukuran file sangat besar maka lebih baik tidak menggunakan `read()`, melainkan menggunakan teknik loop

Selama looping juga dapat melakukan manipulasi terhadap file tersebut, misalnya menangkap atau menampilkan bagian dari string. Seperti pada contoh file `mbox`, saat looping kita dapat menampilkan hanya kalimat yang diawali dengan "tanggal" saja, yaitu "Date :". Berikut contoh penerapannya dalam python:



```
mbox-short.txt X  Mikey.py X
Mikey.py > ...
1  handle = open('mbox-short.txt')
2  count = 1
3  for line in handle:
4      if line.startswith("Date:") and count <= 10:
5          count += 1
6          print(line)
```

Gambar 2.1: Contoh source code menampilkan teks dengan awalan kata yang dituju

Program di atas menghasilkan output 10 baris yang berupa tanggal saja. Berikut outputnya.

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\Mikey.py"
Date: Sat, 5 Jan 2008 09:12:18 -0500

Date: 2008-01-05 09:12:07 -0500 (Sat, 05 Jan 2008)

Date: Fri, 4 Jan 2008 18:08:57 -0500

Date: 2008-01-04 18:08:50 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 16:09:02 -0500

Date: 2008-01-04 16:09:01 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 15:44:40 -0500

Date: 2008-01-04 15:44:39 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 15:01:38 -0500

Date: 2008-01-04 15:01:37 -0500 (Fri, 04 Jan 2008)

PS D:\New folder - Copy\Coding>
```

Gambar 2.2: Output dari source code penggunaan startswith()

Baris kosong muncul karena setiap baris dari file sudah mengandung newline, lalu print menambahkan newline lagi sehingga jadi dobel. Menggunakan `rstrip()` bisa untuk menghapus newline dari data, atau gunakan `print` tanpa menambah newline.

Penyimpanan File

Menulis file di Python sama seperti membuka file, tetapi pakai metode `w` (write), misalnya `open('output.txt','w')`. Memakai `write()` untuk menulis string ke file, lalu tutup dengan `close()`. Contoh lengkap adalah sebagai berikut:

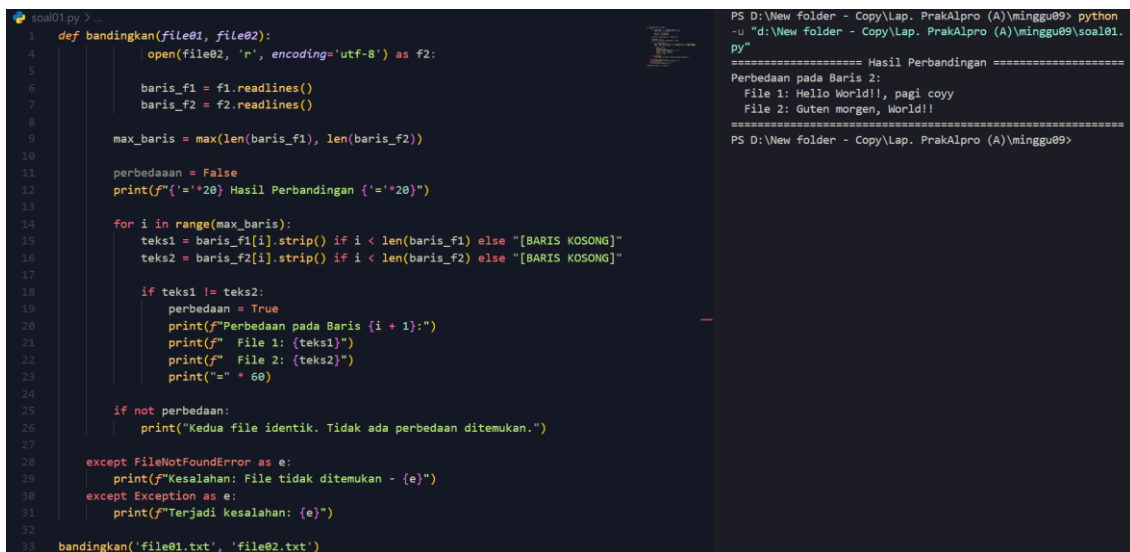
```
1  handle = open('output.txt','w')
2  tulisan = "teks ini akan dituliskan ke file\n"
3  handle.write(tulisan)
4  handle.close()
```

Gambar 2.3: Contoh source code lengkap nya penggunaan metode `w` (write)

LINK GITHUB

: <https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git>

SOAL 01



```
soal01.py > ...
1 def bandingkan(file01, file02):
2     open(file02, 'r', encoding='utf-8') as f2:
3
4         baris_f1 = f1.readlines()
5         baris_f2 = f2.readlines()
6
7         max_baris = max(len(baris_f1), len(baris_f2))
8
9         perbedaan = False
10        print(f"{' '*20} Hasil Perbandingan {' '*20}")
11
12        for i in range(max_baris):
13            teks1 = baris_f1[i].strip() if i < len(baris_f1) else "[BARIS KOSONG]"
14            teks2 = baris_f2[i].strip() if i < len(baris_f2) else "[BARIS KOSONG]"
15
16            if teks1 != teks2:
17                perbedaan = True
18                print(f"Perbedaan pada Baris (i + 1):")
19                print(f"File 1: {teks1}")
20                print(f"File 2: {teks2}")
21                print("=" * 60)
22
23            if not perbedaan:
24                print("Kedua file identik. Tidak ada perbedaan ditemukan.")
25
26        except FileNotFoundError as e:
27            print(f"Kesalahan: File tidak ditemukan - {e}")
28        except Exception as e:
29            print(f"Terjadi kesalahan: {e}")
30
31        bandingkan('file01.txt', 'file02.txt')
```

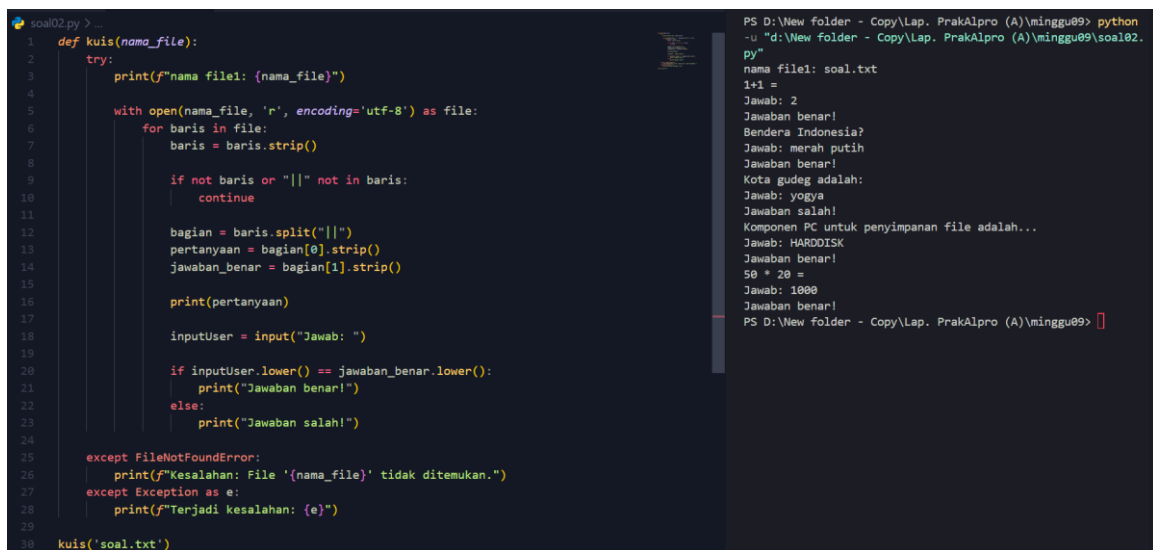
```
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu09> python
-u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu09\soal01.
py"
===== Hasil Perbandingan =====
Perbedaan pada Baris 2:
File 1: Hello World!!, pagi coy
File 2: Guten morgen, World!!
=====
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu09>
```

Gambar 2.4: Source code soal 01

Penjelasan:

Ini adalah program yang dapat membandingkan 2 buah file teks dan kemudian menampilkan perbedaan antar kedua teks per barisnya jika ada perbedaan. Pertama kode program diawali dengan fungsi **bandingkan(file01, file02)** ini untuk membandingkan isi dua file teks dengan cara membuka keduanya dalam mode baca menggunakan **encoding UTF-8** (standar pengkodean karakter yang mampu merepresentasikan hampir semua karakter di dunia) di dalam blok **try** untuk menangani kemungkinan error, isi masing-masing file dibaca menggunakan **readlines()** lalu disimpan dalam bentuk list, kemudian jumlah baris terbanyak ditentukan dengan **max()** agar semua baris tetap dibandingkan meskipun panjang file berbeda, setelah itu variabel **perbedaan** diinisialisasi sebagai False dan program mencetak header, lalu dilakukan perulangan untuk setiap baris di mana teks dari masing-masing file diambil (menggunakan **.strip()** untuk menghapus spasi atau newline, atau diganti dengan "[BARIS KOSONG]" jika salah satu file sudah habis), kemudian kedua teks dibandingkan dan jika berbeda maka **perbedaan** diubah menjadi True serta ditampilkan nomor baris beserta isi dari kedua file dan garis pemisah, sedangkan jika setelah seluruh proses tidak ditemukan perbedaan maka akan ditampilkan bahwa kedua file identik, dan jika terjadi error seperti file tidak ditemukan akan ditangani oleh **FileNotFoundError** atau error lainnya oleh **Exception**, sementara di bagian akhir fungsi dipanggil dengan argumen dua nama file yaitu **'file01.txt'** dan **'file02.txt'**. Lalu output yang dihasilkan seperti pada gambar diatas perbedaanya ada pada baris ke-2 di kedua file.

SOAL 02



```
soal02.py > ...
1 def kuis(nama_file):
2     try:
3         print(f"nama file1: {nama_file}")
4
5         with open(nama_file, 'r', encoding='utf-8') as file:
6             for baris in file:
7                 baris = baris.strip()
8
9                 if not baris or "||" not in baris:
10                     continue
11
12                 bagian = baris.split("||")
13                 pertanyaan = bagian[0].strip()
14                 jawaban_benar = bagian[1].strip()
15
16                 print(pertanyaan)
17
18                 inputUser = input("Jawab: ")
19
20                 if inputUser.lower() == jawaban_benar.lower():
21                     print("Jawaban benar!")
22                 else:
23                     print("Jawaban salah!")
24
25     except FileNotFoundError:
26         print(f"Kesalahan: File '{nama_file}' tidak ditemukan.")
27     except Exception as e:
28         print(f"Terjadi kesalahan: {e}")
29
30 kuis('soal.txt')
```

```
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu09> python
-u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu09\soal02.
py"
nama file1: soal.txt
1+1 =
Jawab: 2
Jawaban benar!
Bendera Indonesia?
Jawab: merah putih
Jawaban benar!
Kota gudeg adalah:
Jawab: yogyakarta
Jawaban salah!
Komponen PC untuk penyimpanan file adalah...
Jawab: HARDISK
Jawaban benar!
50 * 20 =
Jawab: 1000
Jawaban benar!
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu09>
```

Gambar 2.5: Source code soal 02

Penjelasan:

Ini adalah program untuk menampilkan soal sederhana yang diambil dari file teks soal.txt. Pertama kode program diawali dengan fungsi **kuis(nama_file)** yang digunakan untuk membaca file berisi soal dan menjalankan kuis secara interaktif. Di dalamnya terdapat blok **try** untuk menangani kemungkinan error, lalu program mencetak nama file yang digunakan dan membuka file tersebut dalam mode baca. Selanjutnya, file dibaca baris per baris menggunakan perulangan, di mana setiap baris dibersihkan dari spasi atau karakter kosong. Program hanya memproses baris yang tidak kosong dan mengandung pemisah **||**, lalu memisahkan bagian pertanyaan dan jawaban menggunakan **split()**. Pertanyaan ditampilkan ke layar, kemudian user diminta memasukkan jawaban. Jawaban user dibandingkan dengan jawaban benar (tanpa membedakan huruf besar/kecil), dan program akan menampilkan apakah jawaban tersebut benar atau salah. Jika file tidak ditemukan, program akan menampilkan pesan error khusus, dan jika terjadi kesalahan lain, akan ditampilkan pesan error lainnya. Terakhir, fungsi dipanggil dengan file **soal.txt**, yang berisi daftar soal dengan format **"pertanyaan || jawaban"**, sehingga program dapat membaca, menampilkan, dan mengevaluasi jawaban user satu per satu.